

```

void setup() {
    Serial.begin(115200);
    delay(500);

    LOG("SYS", "==== WEATHER GATE v1.2 ===");

    // Инициализация пинов светодиодов и питания
    pinMode(LED_RF_RX, OUTPUT);
    pinMode(LED_MQTT_TX, OUTPUT);
    pinMode(PIN_POWER_SENSE, INPUT);
    pinMode(ETH_POWER_PIN, OUTPUT);
    digitalWrite(LED_RF_RX, LOW);
    digitalWrite(LED_MQTT_TX, LOW);
    digitalWrite(ETH_POWER_PIN, HIGH);

    analogSetAttenuation(ADC_11db);

    // ===== SPI НЕ ИНИЦИАЛИЗИРУЕМ! =====
    // Библиотека rtl_433_ESP сама инициализирует SPI с пинами,
    // которые прописаны в rtl_433_ESP.cpp:
    // SCK=12, MISO=15, MOSI=14, CS=2, GDO0=4

    // ===== I2C ДЛЯ RTC =====
    Wire.begin(I2C_SDA, I2C_SCL);
    if (rtc.begin()) {
        rtc_found = true;
        LOG("RTC", "DS3231 found");
    } else {
        LOG("RTC", "DS3231 NOT found");
    }

    // ===== LITTLEFS =====
    if (!LittleFS.begin(true)) {
        LOG("FS", "LittleFS mount failed");
    } else {
        LOG("FS", "LittleFS mounted");
    }

    // ===== ETHERNET =====
    WiFi.mode(WIFI_OFF);
    btStop();
    ETH.begin(ETH_PHY_LAN8720, 1, 23, 18, 16, ETH_CLOCK_GPIO17_OUT);
    ETH.setHostname(hostname.c_str());
    delay(500);
}

```

```

// Ожидание IP
unsigned long start = millis();
while (ETH.localIP().toString() == "0.0.0.0" && millis() - start < 10000) {
    delay(100);
}
LOG("ETH", "IP address: %s", ETH.localIP().toString().c_str());

// ===== НАСТРОЙКИ =====
loadSettings();

// ===== CC1101 (БИБЛИОТЕКА ВСЁ ДЕЛАЕТ САМА) =====
rf.setCallback(rtl_433_Callback, msgBuffer, 512);
rf.rtlVerbose = 1; // Включаем verbose для отладки
rf.initReceiver(CC1101_GDO0, current_freq);
rf.enableReceiver();
LOG("RF", "CC1101 initialized at %.2f MHz", current_freq);

// ===== MQTT =====
if (mqtt_ip.length() > 5) {
    mqtt.setServer(mqtt_ip.c_str(), 1883);
    mqtt.setCallback(mqttCallback);
    LOG("MQTT", "Server set to %s", mqtt_ip.c_str());
}

// ===== WEB-СЕРВЕР =====
// API endpoints (оставляем как было)
server.on("/api/data", HTTP_GET, []) {
    DynamicJsonDocument doc(1024);
    doc["temperature"] = temperature;
    doc["humidity"] = humidity;
    doc["pressure"] = pressure;
    doc["wind_speed"] = wind_speed;
    doc["wind_gust"] = wind_gust;
    doc["wind_direction"] = wind_direction;
    doc["rain"] = rain;
    doc["temp_min"] = temp_min;
    doc["temp_max"] = temp_max;
    doc["pressure_min"] = pressure_min;
    doc["pressure_max"] = pressure_max;
    doc["wind_max"] = wind_max;
    doc["battery"] = battery_voltage;
    doc["battery_status"] = getBatteryStatus();
    doc["power_source"] = digitalRead(PIN_POWER_SENSE) ? "BAT" : "AC";
}

```

```

doc["mqtt_status"] = mqtt.connected() ? "OK" : "Offline";
doc["hostname"] = hostname;
doc["version"] = CURRENT_VERSION;
doc["uptime"] = millis() / 1000;
doc["radio_errors"] = radio_errors;
doc["radio_success"] = radio_success;
doc["current_freq"] = current_freq;
doc["battery_coeff"] = battery_coeff;
doc["mqtt_ip"] = mqtt_ip;
if (rtc_found) doc["datetime"] = formatDateTime(rtc.now());
String buf;
serializeJson(doc, buf);
server.send(200, "application/json", buf);
});

server.on("/api/set_freq", HTTP_POST, []) {
  if (server.hasArg("f")) {
    float new_freq = server.arg("f").toFloat();
    if (new_freq >= 860.0 && new_freq <= 930.0) {
      current_freq = new_freq;
      pref.begin("weather", false);
      pref.putFloat("freq", current_freq);
      pref.end();
      rf.disableReceiver();
      delay(50);
      rf.initReceiver(CC1101_GDO0, current_freq);
      rf.enableReceiver();
      LOG("RF", "Frequency changed to %.2f MHz", current_freq);
      server.send(200, "text/plain", "OK");
    } else {
      server.send(400, "text/plain", "Invalid frequency");
    }
  } else {
    server.send(400, "text/plain", "Missing frequency");
  }
};

server.on("/api/reset_errors", HTTP_POST, []) {
  radio_errors = 0;
  radio_success = 0;
  saveCounters();
  server.send(200, "text/plain", "OK");
};

```

```

server.on("/api/config", HTTP_POST, []() {
    String body = server.arg("plain");
    DynamicJsonDocument doc(1024);
    if (!deserializeJson(doc, body)) {
        if (doc.containsKey("hostname")) hostname = doc["hostname"].as<String>();
        if (doc.containsKey("mq_ip")) mqtt_ip = doc["mq_ip"].as<String>();
        if (doc.containsKey("mq_u")) mqtt_user = doc["mq_u"].as<String>();
        if (doc.containsKey("mq_p")) mqtt_password = doc["mq_p"].as<String>();
        if (doc.containsKey("bat_k")) battery_coeff = parseSafeFloat(doc["bat_k"].as<String>());
        ETH.setHostname(hostname.c_str());
        mqtt.setServer(mqtt_ip.c_str(), 1883);
        saveAll();
        server.send(200, "OK");
        delay(500);
        ESP.restart();
    } else {
        server.send(400, "text/plain", "JSON Error");
    }
});

server.onNotFound([]() {
    if (!handleFileRead(server.uri())) {
        server.send(404, "text/plain", "Not Found");
    }
});

server.begin();
LOG("HTTP", "Server started");

// Инициализация истории
for (int i = 0; i < POINTS_HISTORY; i++) {
    history.temp[i] = 0;
    history.pressure[i] = 0;
    history.wind[i] = 0;
}

lastSaveTime = millis();
lastHistoryTime = millis();

LOG("SYS", "Setup complete, waiting for data...");
}

```